

Verteilte Systeme

Praktikum: Sturmflut Nordsee

Lars-Olof Burchard und Ronald Moore

Hochschule Darmstadt
Fachbereich Informatik
Sommersemester 2026

1 Übersicht

In diesem Praktikum entwickeln Sie ein verteiltes System für den Katastropheneinsatz auf einer *fiktiven* Nordseeinsel. Nach einer Sturmflut müssen Rettungs-, Reparatur- und Versorgungsfahrzeuge koordiniert werden. Alle Teilprozesse laufen in eigenen Containern. Eine Leitstelle verwaltet das System, stationäre Sensoren und Kameras melden Vorfälle, und ein Dashboard zeigt die aktuelle Lage. Im Mittelpunkt stehen heterogene Datenquellen, unterschiedliche Fahrzeugrollen und knappe Infrastruktur. Drohnen und Rover arbeiten batteriebetrieben und müssen deshalb regelmäßig Ladestationen anfahren, deren Kapazität begrenzt ist. Einsätze entstehen direkt aus Sensorwerten, z. B. aus Pegelständen, Personenerkennung, Strukturwerten oder Lagerbeständen.

Schritt für Schritt realisieren Sie das Gesamtsystem mit unterschiedlichen Kommunikationsmechanismen:

1. Registrierung von Infrastruktur, Sensorik und Fahrzeugen über eine selbst implementierte REST-API mit TCP-Sockets
2. Einsatzvergabe an unterschiedliche Fahrzeugtypen mittels RPC
3. Entkoppelte Ereignisverarbeitung von Kameras, Sensoren und Fahrzeugen über MQTT
4. Dezentrale Koordination für den Zugriff auf knappe Infrastruktur mit einem Verfahren aus der Vorlesung

Ziel ist der saubere Entwurf, die Implementierung, das Testen und das Deployen eines verteilten Systems über ein ganzes Semester.

2 Grundregeln

Folgende Regeln gelten für das gesamte Praktikum und für alle Aufgaben:

- Die Praktikumsaufgaben werden nach Möglichkeit in Zweier-Teams bearbeitet. Klären Sie frühzeitig Erwartungen, Arbeitsweise und Erreichbarkeit.

- Die Aufgaben müssen außerhalb der Praktikumstermine vor- und nachbereitet werden. Die Bearbeitungsdauer ist ein ganzes Semester. Rechnen Sie mit kontinuierlicher Arbeit.
- Jede Gruppe verwendet ausschließlich das bereitgestellte Repository im Git-Lab der H-DA. Die Lösungen müssen im Main-Branch verfügbar sein.
- Es ist ein Build-Tool zu verwenden, z. B. Make, Maven, Gradle, Cargo oder cmake.
- Die Lösungen müssen containerisiert und mittels Docker und Docker-Compose zu testen sein.
- Jede Komponente (jeder Teilprozess) des verteilten Systems muss zwingend in einem eigenen Container laufen. Dies gilt insbesondere für Leitstelle, Broker, Dashboard, Sensor-Gateway und jeden Fahrzeugtyp. Eine Zusammenfassung mehrerer Teilprozesse in einem Container ist nicht zulässig.
- Es sind keine Programmiersprachen vorgegeben. Eine Kombination mehrerer Sprachen ist erlaubt.
- Sockets und HTTP müssen in der entsprechenden Aufgabe nativ, d. h. ohne HTTP-Server-Frameworks oder externe REST-Bibliotheken, implementiert werden. Für RPC, MQTT, Logging, Konfiguration und Serialisierung dürfen frei wählbare Bibliotheken eingesetzt werden.
- Alle Teammitglieder müssen die Architektur, die Kommunikationsflüsse und die getroffenen Designentscheidungen in der Abnahme erklären können.

3 Testdokumentation

Tests müssen einheitlich dokumentiert werden, damit Zweck, Setup und Ergebnis für die Abnahme nachvollziehbar sind.

3.1 Pflichtumfang

- Zu jeder Aufgabe ist mindestens ein funktionaler und ein nicht-funktionaler Test zu dokumentieren.
- Jeder Test muss reproduzierbar sein. Container, Startbedingungen, Eingaben und erwartete Ergebnisse sind anzugeben.
- Die Ergebnisse der Tests müssen dokumentiert werden. Für jede Aufgabe außer Aufgabe 0 soll ein Testprotokoll abgegeben werden.
- In jedem Testprotokoll sollen:
 - die neuen Tests für alle neuen Komponenten dokumentiert werden;

- die alten Tests für alle alten Komponenten wiederholt werden, die neue Ergebnisse dokumentiert, und die Entwicklung der Ergebnisse analysiert werden.

Infolgedessen soll im Lauf des Semesters die Länge der Testprotokolle immer wachsen.

- Die Testprotokolle sollen im Repository gespeichert werden.
- Die Testdokumentation soll versioniert im Repository liegen.

3.2 Verbindliches Testschema

Für jeden Test ist das folgende Schema auszufüllen:

- **Aufgabe / Meilenstein:** Zu welcher Aufgabe des Praktikums gehört der Test?
- **Testtyp:** Funktional oder nicht-funktional
- **Titel:** Kurze, sprechende Bezeichnung des Tests
- **Ziel:** Welche Eigenschaft oder Anforderung wird überprüft?
- **Vorbedingungen / Setup:** Container, Konfiguration, Testdaten
- **Durchführung:** Durchführung in nachvollziehbaren Schritten
- **Erwartetes Ergebnis:** Welches beobachtbare Verhalten gilt als korrekt?
- **Tatsächliches Ergebnis:** Was wurde bei der Ausführung tatsächlich beobachtet?
- **Artefakte / Nachweise:** Logs, Screenshots oder Reports
- **Bewertung:** PASS, FAIL oder BLOCKED
- **Zugehöriger Commit / Versionsstand:** Auf welchem Stand wurde der Test durchgeführt?

3.3 Hinweise zur Qualität

- Nicht-funktionale Tests sollen messbar sein.
- Fehlgeschlagene Tests sind kurz zu kommentieren und die Probleme sind anschliessend zu beheben, bis der Test erfolgreich ist.
- Sowohl manuelle als auch automatisierte Tests sind möglich.

4 Aufgabenstellung

Sie entwickeln das System schrittweise. Im Fokus stehen Kommunikation, Architektur, Tests und ein reproduzierbarer Betrieb in Containern.

4.1 Aufgabe 0 - Vorbereitung und Architektur

Der erste Praktikumstermin dient der Vorbereitung des Semesters. Richten Sie Entwicklungsumgebung, Docker, Docker Compose und Repository ein. Erstellen Sie eine kurze Anforderungsanalyse für die Aufgaben 1 bis 4 und leiten Sie daraus eine erste Container- und Komponentenarchitektur ab.

4.2 Aufgabe 1 - Leitstelle und Registrierung via Sockets, HTTP und REST

Im ersten technischen Schritt implementieren Sie die Leitstelle als zentrale Verwaltungseinheit. Die REST-Schnittstelle ist manuell über TCP-Sockets zu implementieren. Externe HTTP-Bibliotheken oder Webframeworks sind nicht erlaubt.

Die Leitstelle verwaltet eine Inselkarte als Grid mit mindestens 20x20 Feldern, der darin gespeicherte Status kann von einem Webbrowser ausgelesen werden. Dieses Dashboard wird von der Leitstelle jederzeit über HTTP zur Verfügung gestellt. Die Karte kann dynamisch bei Start erzeugt oder aus einer Datei geladen werden. Anforderungen:

- HTTP-Server: Der Server muss mindestens GET- und POST-Anfragen korrekt parsen und beantworten. Nicht unterstützte Methoden sollen mit einem passenden Fehlercode beantwortet werden.
- Der HTTP Server muss Web-Seiten produzieren, die von einem gängigen Browser (z.B. Chrome oder Firefox) gerendert werden können.
- Registrierung: Fahrzeuge, Kameras und Sensorquellen registrieren sich über die REST-Schnittstelle.
- REST-API: Implementieren Sie mindestens geeignete Endpunkte für Registrierung, Erzeugung von Vorfällen, Abfrage der Karte und Abfrage des Gesamtzustands. Beispiele wären POST /unit, POST /incident, GET /map und GET /status.
- Sensordaten: Mindestens zwei unterschiedliche Sensortypen müssen rohe Messwerte liefern, z. B. `water_level_cm`, `vibration_index`, `supply_level_percent` oder `person_confidence`.
- Infrastruktur: Die Karte muss feste Infrastruktur enthalten, z. B. Brücke, Hafen, Ladestation, Landefeld oder Depot.
- Jedes Grid-Feld ist entweder Wasser oder Land.

Zusätzlich implementieren Sie mindestens zwei Kameras oder andere Sensorquellen, die Vorfälle wie `blocked_route`, `person_detected` oder `water_level_alert` meldet. Diese Vorfälle müssen aus konkreten (simulierten) Messwerten oder Schwellwertverletzungen hergeleitet werden.

4.3 Aufgabe 2 - Einsatzvergabe via Remote Procedure Call (RPC)

Nun werden spezialisierte Einsatzfahrzeuge hinzugefügt. Verwenden Sie mindestens zwei Rollen, z. B. *Einsatzdrohnen*, *Reparaturfahrzeuge* oder *Versorgungsfahrzeuge*. Die Fahrzeugtypen sollen sich sichtbar unterscheiden, z. B. Drohnen dürfen über Wasser fliegen, Bodenfahrzeuge nicht. Die Leitstelle weist Einsätze per Remote Procedures (RPC) zu.

Apache Thrift oder Google RPC (gRPC) dürfen verwendet werden.

Anforderungen:

- IDL-Definition: Definieren Sie eine RPC-Schnittstelle, über die die Leitstelle einem Fahrzeug einen Einsatz zuweisen kann. Ein Einsatz soll mindestens eine Einsatz-ID, einen Typ, eine Zielposition und eine Priorität enthalten.
- Registrierung: Die Fahrzeuge registrieren sich weiterhin über die REST-Schnittstelle aus Aufgabe 1 bei der Leitstelle.
- Heterogene Rollen: Die Leitstelle darf nicht jedes Fahrzeug für jeden Einsatz verwenden. Die Auswahl muss vom Ereignistyp abhängen.
- Rollenzuordnung: Personenerkennung oder schwer erreichbare Orte werden einer Einsatzdrohne zugewiesen. Struktur- oder Wegschäden werden einem Reparatur-Rover zugewiesen. Niedrige Lagerbestände oder Materialbedarf werden einem Versorgungs-Rover zugewiesen.
- Rückmeldung: Das Fahrzeug bestätigt den Auftrag, simuliert Anfahrt und Bearbeitung und meldet Abschluss oder Fehler an die Leitstelle zurück.
- Aufgabenverhalten: Die Fahrzeuge sollen je Rolle ein eigenes Verhalten simulieren, z. B. inspizieren, reparieren oder liefern.
- Zustände: Für Fahrzeuge sollen mindestens die Zustände `IDLE`, `ASSIGNED`, `BUSY` und `ERROR` unterscheidbar sein.
- Dashboard: Der aktuelle Arbeitsfortschritt muss über die vorhandenen REST-Endpunkte sichtbar sein.

4.4 Aufgabe 3 - Ereignisse und Telemetrie über MoM (MQTT)

In dieser Aufgabe werden neue Vorfälle und Statusmeldungen entkoppelt über MQTT oder eine vergleichbare Pub/Sub-Technologie verarbeitet. Leitstelle und Dashboard leiten ihre Sicht auf das System aus diesen Nachrichten ab. Im Zentrum stehen nun stationäre Ereignisquellen statt nur mobile Teilnehmer.

Wir empfehlen die Verwendung von MQTT.

Anforderungen:

- Middleware: Setzen Sie einen Broker sowie passende Clients für Ihre Komponenten ein. Der Broker läuft als eigener Container.

Hinweis: Es gibt verschiedenen Broker, die Sie verwenden können, u.a. Mosquitto, HiveMQ und EMQX. Sie sollen in keinem Fall Ihren eigenen Broker programmieren.

- Publisher/Subscriber:
 - Kameras und Sensoren publizieren Ereignisse; Fahrzeuge publizieren Telemetrie wie Position, Fortschritt und Fehler.
 - Ereignisverarbeitung: Die Leitstelle soll Duplikate, alte Nachrichten oder mehrere Meldungen zur gleichen Position sinnvoll behandeln. Fahrzeuge sollen autonom auf Ereignisse reagieren, z. B. Gefahren vermeiden.
- Topics und QoS: Definieren Sie eine Topic-Struktur und passende Quality-of-Service-Stufen.
- Ereignisdefinition: Dokumentieren Sie, wie aus Sensorwerten konkrete Einsatztypen entstehen, z. B. `water_level_cm > 80` erzeugt `water_level_alert` oder `supply_level_percent < 20` erzeugt `resupply_required`.
- Nachrichtenformat: Das Format der Nachrichten, z. B. JSON, muss klar definiert und dokumentiert werden.
- REST-Integration: Die Endpunkte `GET /map` und `GET /status` sollen weiterhin verfügbar sein, ihre Datenbasis soll nun jedoch primär aus den MQTT-Nachrichten stammen.
- Fehlerszenarien: Testen Sie Ausfall oder Neustart einzelner Publisher und des Brokers.

4.5 Aufgabe 4 - Verteilte Koordination mit Ricart/Agrawala

Im letzten Schritt soll eine zentrale Engstelle aus dem System entfernt werden. Die Fahrzeuge koordinieren den exklusiven Zugriff auf eine knappe Ressource selbst. Im Standardszenario ist diese Ressource eine Ladestation mit begrenzter Kapazität, die von batteriebetriebenen Drohnen und Rover regelmäßig benötigt wird. Sie können sich weitere Szenarien/Ressourcen überlegen.

Verwenden Sie hierfür den Algorithmus von Ricart/Agrawala aus der Vorlesung. Die Leitstelle darf den Zugriff nur noch beobachten und visualisieren, aber nicht mehr zentral vergeben.

Anforderungen:

- Verteiltes Ausschlussverfahren: Mindestens drei Fahrzeuge müssen am gegenseitigen Ausschluss beteiligt sein. Die Fahrzeuge koordinieren sich untereinander mit REQUEST- und REPLY-Nachrichten gemäß Ricart/Agrawala.
- Infrastruktur statt Task-Wahl: Koordiniert wird nicht die Vergabe eines Einsatzes, sondern der Zugriff auf die Ladestation.
- Ordnung: Die Reihenfolge konkurrierender Zugriffe muss über logische Zeit mit Totalordnung nachvollziehbar sein.
- Kommunikation: Wählen Sie für die Nachrichten des Algorithmus einen zu Ihrer Architektur passenden Kommunikationsweg. Begründen Sie die Entscheidung.
- Safety: Zeigen Sie experimentell, dass niemals zwei Fahrzeuge gleichzeitig die kritische Ressource nutzen.
- Liveness: Zeigen Sie, dass wartende Fahrzeuge nach Abschluss anderer Zugriffe weiterarbeiten können.
- Fehlerbetrachtung: Zeigen Sie, was bei Nachrichtenausfall, Prozessabsturz oder nicht-FIFO-Kommunikation passiert.

5 Pi-Lab

Sofern das Pi-Lab in diesem Semester zur Verfügung steht, können Sie Ihre Lösung dort in einer realen verteilten Umgebung deployen. Die Nutzung des Pi-Labs ist ausdrücklich empfohlen, aber keine formale Zusatzleistung.

Beachten Sie insbesondere, dass Ihre Container und Images auf einer ARM-Architektur lauffähig sein müssen. Planen Sie dies frühzeitig in Ihrer Build- und Deploy-Strategie ein.